

## FICHE 08 : INTERFACES, ITERABLE ET CLASSES ABSTRAITES

### objectif

- Pouvoir créer, implémenter et utiliser une interface.
- Découvrir et concevoir des classes et des méthodes abstraites.

### Vocabulaire

|           |             |          |                  |                   |
|-----------|-------------|----------|------------------|-------------------|
| interface | implémenter | Iterable | classe abstraite | méthode abstraite |
|-----------|-------------|----------|------------------|-------------------|

### Exercices

Créez, dans le répertoire APOO, un projet intitulé **APOO\_fiche8** afin d'y implémenter les classes demandées dans cette fiche.

#### 1. Solides

- a) On désire créer une interface pour les solides. Chaque solide devra pouvoir effectuer les opérations suivantes :

- Donner son volume.
- Donner sa surface.

Trois classes implémenteront cette interface : la classe `Sphere`, la classe `Cylindre` et la classe `Cube`.

#### Représentez cette interface et ces classes en UML.

Songez aux attributs nécessaires pour ces classes.

N'oubliez pas de mettre un constructeur, des getters pour les attributs et une méthode `toString` dans chaque classe.

- b) Implémentez cette interface et ces classes en Java.

Remarque : Une dimension d'un solide doit être strictement supérieure à 0. Les constructeurs des différentes classes doivent lancer une `IllegalArgumentException` si on leur fournit une dimension pour un solide inférieure ou égale à 0.

Rappels :

- Volume d'une sphère :  $4 \pi r^3 / 3$  où  $r$  est le rayon
- Surface d'une sphère :  $4 \pi r^2$  où  $r$  est le rayon
- Volume d'un cylindre :  $\pi r^2 h$  où  $h$  est la hauteur et  $r$  le rayon
- Surface d'un cylindre :  $2 \pi r^2 + 2 \pi r h$

- c) On veut maintenant que deux cubes soient égaux s'ils ont le même côté, que deux sphères soient égales si elles ont le même rayon et que deux cylindres soient égaux s'ils ont la même hauteur et le même rayon. Modifiez l'UML et l'implémentation afin de prendre en compte cela.
- d) On désire maintenant créer une classe `ListeSolides` qui gardera une liste de solides. Il faudra aussi pouvoir effectuer les opérations suivantes :
- dire si la liste est vide ;

- donner le nombre de solides qu'il y a dans la liste;
- ajouter un solide dans la liste. Si le solide est déjà présent dedans, celui-ci ne sera pas ajouté ;
- supprimer un solide de la liste ;
- vérifier si un solide se trouve dans la liste ;
- pouvoir parcourir la liste des solides (avec un foreach) ;
- trouver le(s) solide(s) offrant le plus grand volume.

Il faudra aussi écrire la méthode `toString` dans `ListeSolides` afin qu'elle liste tous les `Solide` présents dans la liste (un solide par ligne).

Faites le diagramme UML de cette classe.

- e) Une fois le diagramme de classe de `ListeSolides` validé, implémentez cette classe en java. Les méthodes doivent lancer une `IllegalArgumentException` en cas de paramètre invalide.
- f) Récupérez la classe `TestListeSolides` sur moodle et adaptez-la pour qu'elle compile avec vos classes. Exécutez `TestListeSolides` et vérifiez que vous obtenez bien l'affichage attendu (fourni dans le fichier `affichage_TestListeSolides`).
- g) Complétez la classe `TestListeSolides` en ajoutant de nouveaux solides à votre liste. Ensuite, affichez :
  - tous les solides dont le volume est supérieur à 200
  - tous les solides dont la surface est comprise entre 100 et 500
  - ...

## 2. Contrôle technique

Un organisme de contrôle gère les contrôles techniques des véhicules. On distingue deux types de véhicules : les voitures et les utilitaires. Tout véhicule possède un numéro de châssis qui permet de l'identifier, une immatriculation, une date de mise en circulation, une date de dernier contrôle technique et le kilométrage. Toutes ces informations sont consultables. Cependant, on ne doit pouvoir modifier que l'immatriculation, le kilométrage et la date du dernier contrôle technique.

Il faut aussi pouvoir, pour tout véhicule, dire s'il est en ordre de contrôle technique mais cela se fera de façon différente selon son type :

- Une voiture est en ordre si elle a été mise en circulation il y a moins de 4 ans ou si le dernier contrôle technique a eu lieu il y a moins d'un an.
- Un utilitaire est en ordre si le nombre de kilomètres qu'il a parcouru depuis son dernier contrôle technique est inférieur au nombre maximum de kilomètres qu'il peut faire entre deux contrôles techniques. Pour pouvoir vérifier qu'un utilitaire est en ordre, il faut donc retenir le nombre de kilomètres qu'il peut faire entre deux contrôles techniques ainsi que son kilométrage lors de son dernier contrôle technique. Ces deux informations doivent être consultables mais seul le kilométrage au dernier contrôle technique sera modifiable

Un véhicule est neuf lorsqu'il est enregistré auprès du contrôle technique. De ce fait, lors de la création d'un véhicule, il ne faut pas préciser le (ou les) kilométrages qui seront initialisés à 0, ni la date du dernier contrôle technique qui sera initialisée à la date de mise en circulation. Toutes les informations nécessaires à la création d'un véhicule sont fournies lors

de sa création. Cependant, pour un utilitaire, il doit être possible d'en créer un en précisant ou non le nombre de kilomètres qu'il peut faire entre deux contrôles techniques. Dans le cas où il n'est pas précisé, il sera initialisé à la valeur par défaut de 50000.

- a) Faites un diagramme de classes en vous basant sur les informations qui précèdent. Pour les dates, vous ne devez pas créer votre propre classe `Date` mais utiliser la classe existante `LocalDate` du package `java.time`.
- a) Une fois votre diagramme validé, implémentez-le en java. Vous devez lancer une `IllegalArgumentException` en cas de paramètres invalides sachant que :
  - Un paramètre ne peut pas être null.
  - Une chaîne de caractères ne peut pas être vide.
  - Un nouveau kilométrage doit être strictement supérieur à l'ancien.
  - Une nouvelle date de dernier contrôle technique doit être ultérieure à l'ancienne

Remarques :

- Pour plus d'informations à propos des `LocalDate`, consultez la javadoc : <https://docs.oracle.com/en/java/javase/17/docs/api/>
- N'oubliez pas de tester vos classes !

### 3. Magasin en ligne

Un magasin désire créer un site web pour vendre ses différents produits. Pour l'instant, il ne vend que trois types de produits : CDs, DVDs et livres. Un produit est toujours identifié par sa référence et possèdera toujours un prix.

De plus, un *CD* possède un *titre*, un *artiste* et un *nombre de morceaux* . Un *DVD* possède un *titre* et un *réalisateur*. Un *Livre* possède un *titre*, un *auteur* et un *nombre de pages*.

Un *Panier* permet de contenir les produits qu'un client désire acheter. La classe *Panier* permettra de faire les opérations suivantes :

- ajouter un produit ;
  - supprimer un produit (qui supprime un exemplaire du produit s'il est présent) ;
  - donner le nombre de produits contenus dans le panier ;
  - calculer le prix total du panier ;
  - une méthode `toString()` permettant de lister les différents produits contenus dans le panier ;
  - parcourir les produits présents dans le panier (avec un `foreach`) ;
  - retrouver un produit selon sa référence.
- a) Représentez, en UML, les classes `Cd`, `Dvd`, `Produit`, `Livre` et `Panier` sachant que toutes les informations des produits doivent être consultables et que seul le prix doit être modifiable.
  - b) Implémentez ces classes en Java. Vous ne devez pas gérer les exceptions.
  - c) Implémentez une classe `TestPanier`. Dans ce test, créez un panier et ajoutez-y deux livres et un DVD. Ensuite affichez son prix ainsi que les produits qui s'y trouvent. Finalement, affichez le nombre de produits contenus dans ce panier.

## 4. Agence de voyage

Une agence de voyage propose différentes formules de vacances : la formule simple, les séjours, les circuits et les croisières. Pour chaque formule, il faut garder une date de départ et une durée (nombre de nuits). Il faut en outre pouvoir connaître le prix mais la façon de le trouver variera d'une formule à l'autre.

Pour la formule simple, on gardera le vol aller et le vol retour. Le prix est égal à la somme des prix des deux vols.

Pour un vol, il faudra garder, la date et l'heure du vol, son prix, son numéro, l'aéroport de départ et l'aéroport d'arrivée. Un vol peut être identifié par son numéro associé à sa date et son heure.

Les séjours sont en fait des formules simples pour lesquelles on a en plus une réservation d'hôtel (pour une réservation, on gardera le nom de l'hôtel, le nombre d'étoiles de l'hôtel, le prix pour une nuit). Le prix sera donné par le prix de la formule simple auquel il faudra rajouter le prix de l'hôtel.

Pour les circuits et les croisières, il faudra garder leur prix, leur nom et une liste d'étapes, une étape étant constituée d'une ville et d'une date. Il faudra aussi pouvoir :

- ajouter une étape à condition qu'il n'y a pas déjà une étape à la même date et que sa date est bien comprise entre la date de départ et de fin du voyage ;
- retirer une étape ;
- parcourir les étapes (avec un foreach).

Pour les circuits, il faudra en plus garder une description du circuit tandis que pour les croisières, il faudra en plus garder les informations du bateau c.-à-d. son nom, son nombre de cabines et les activités qu'on peut y pratiquer. Pour le bateau, on doit en plus pouvoir ajouter et supprimer une activité. On ne peut pas ajouter deux fois la même activité.

a) Faites un diagramme de classes en vous basant sur les informations qui précèdent et sachant que :

- Toutes les classes posséderont un constructeur recevant en paramètre toutes les informations **nécessaires** afin d'initialiser ses attributs.
- Les méthodes pouvant échouer doivent le signaler par un retour adéquat.
- Vous ne devez pas mettre les getters et les toString.

b) Une fois votre diagramme validé, implémentez-le en java sachant que :

- Il y aura des getters pour tous les attributs autres que des listes et autres que le prix de la formule simple, d'un circuit ou d'une croisière.
- Toutes les classes ont une méthode toString permettant de récupérer la totalité de l'état.

Dans un premier temps, ne testez pas la validité des paramètres et ne lancez pas d'exceptions !

Remarque : n'oubliez pas de tester vos classes !

c) Bonus : testez les paramètres et lancer une `IllegalArgumentException` en cas de paramètres invalides.